

Daniel Hernandez

CSCI-611

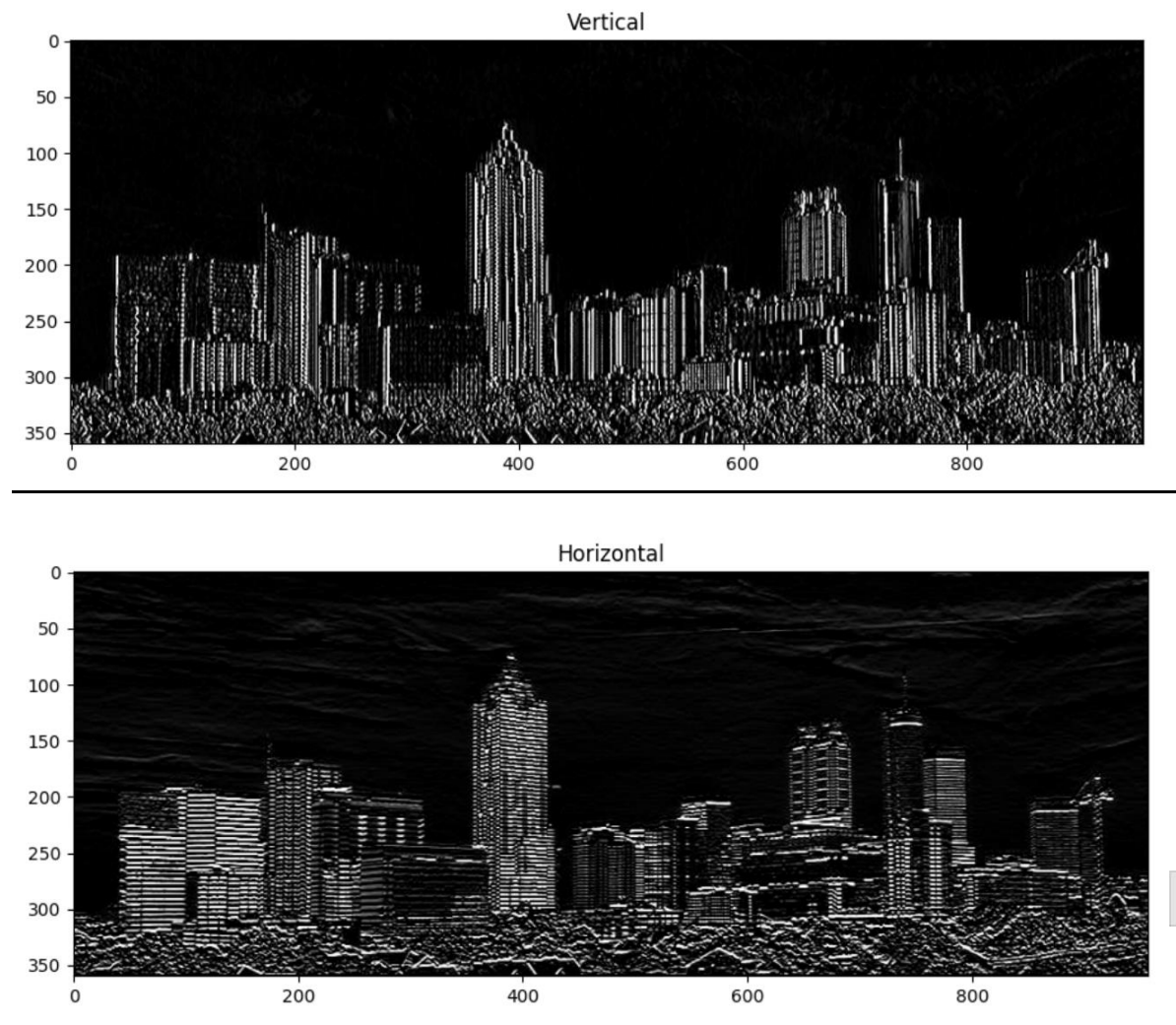
Summer 2025

Bo Shen

Assignment 2; Part 1

Sobel Operator Edge Detector

Using the Sobel operators defined in the assignment prompt, the images below reflect the changes to the image with highlighted vertical and horizontal edges



Corner Detection

Surprisingly, the effect of the following corner detection kernels, which were given in the lecture slides, did not produce visually significant changes to the image.

```
# corner detection kernels

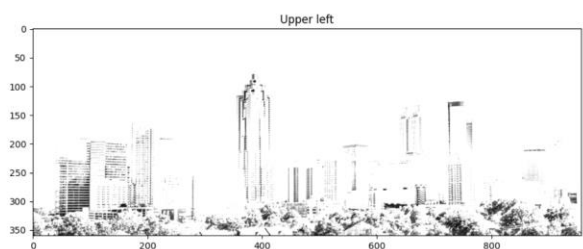
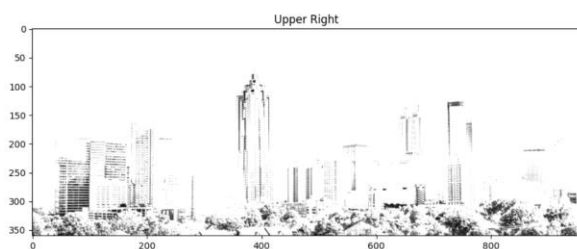
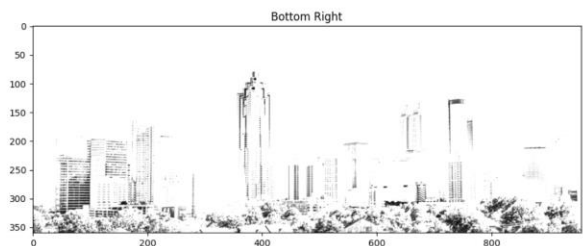
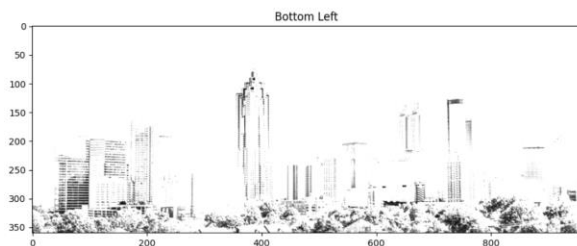
bottomLeft = np.array([[ 0, 1, 0],
                       [ 0, 1, 1],
                       [ 0, 0, 0]])

bottomRight = np.array([[ 0, 1, 0],
                        [ 1, 1, 0],
                        [ 0, 0, 0]])

upperRight = np.array([[ 0, 0, 0],
                       [ 1, 1, 0],
                       [ 0, 1, 0]])

upperLeft = np.array([[ 0, 0, 0],
                      [ 0, 1, 1],
                      [ 0, 1, 0]])

fig = plt.figure(figsize=(24,24))
```



Scaling/Blurring

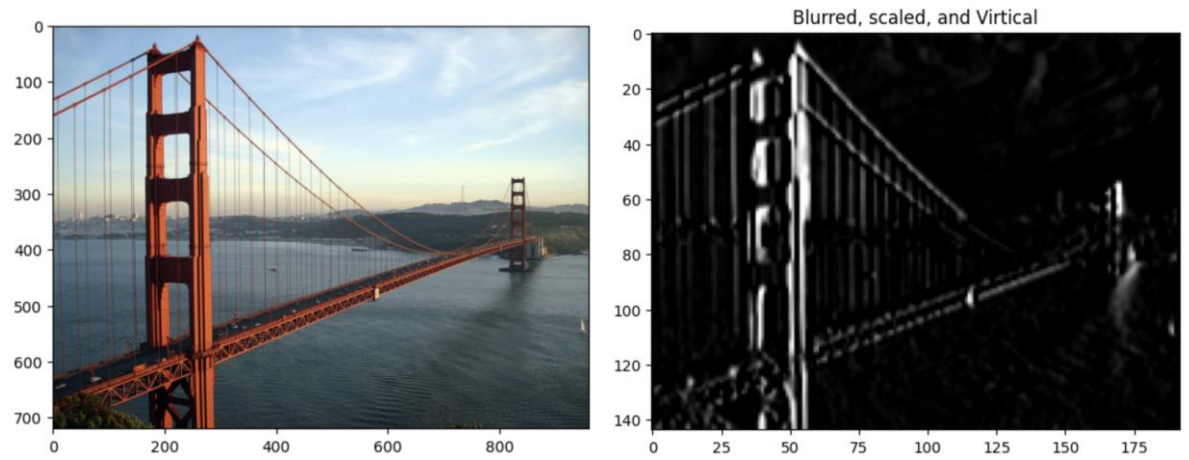
Here, I am dividing each neuron in the 2x2 kernel by the total amount of neurons in the kernel in order to average out each neuron in the image. This produces the blurring effect since the value of each pixel is changed to the average of the entire kernel. In the next line, we are

producing the scaling effect by specifying that we only want every 50th column and every 50th row from the original image to appear in the scaled down version.

```
blurred_image = cv2.filter2D(gray, -1, S2x2/4.0)
```

```
shrink = blurred_image[::50, ::50]
```

Challenge



Here I demonstrate a blurred, scaled, and vertically filtered image. Using a 10x10 kernel to blur, choosing every 5 pixel to scale down, and applying the vertical Sobel operator edge detector (kx), I am able to transform my custom image.

```
S10x10 = np.array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                    [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])
```

```
# Filter the image using filter2D, which has inputs: (grayscale image, number of channels, kernel)
blurred_image = cv2.filter2D(gray3, -1, S10x10/100.0) # blurring
```

```
shrink = blurred_image[::5, ::5] #scaling down
```

```
virt_image = cv2.filter2D(shrink, -1, kx) #using sobel virt kernel
```

Daniel Hernandez

CSCI-611

Summer 2025

Bo Shen

Assignment 2; Part 2

This is a report on the CNN model developed to classify images using the CIFAR-10 dataset. This model was built using PyTorch and tuning included multiple stages

Comparing 2 optimizers: Adam and SGD

optimizer	lr	accuracy
adam	0.1	10
	0.01	10
	0.001	67
	0.0001	55
SGD	0.1	60
	0.01	56
	0.001	28
	0.0001	12

The best result from tuning the optimizer came from Adam optimizer with a learning rate of 0.001.

Kernel Tuning

kernel size	padding	accuracy (%)
7, 5, 3	3, 2, 1	69
3, 3, 3	1, 1, 1	71

Initially, the kernel size decreased between layers. However, the best result came from adjusting all kernel sizes to 3x3 which increased the overall accuracy from 69% to 71%.

Layer Tuning

Set up				param	accuracy (%)
1	conv1	3	10		
	conv2	10	30		
	conv3	30	50		67
2	conv1	3	10		
	conv2	10	20		
	conv3	20	30		66
3	conv1	3	10		
	conv2	10	30		
	conv3	30	40		66
4	conv1	3	15		
	conv2	15	30		
	conv3	30	60		67
5	conv1	3	22		
	conv2	22	38		
	conv3	38	75	292129	69
6	conv1	3	25		
	conv2	25	50		
	conv3	50	100	402260	68

Testing 6 combinations of layer parameter inputs, the best result came from set up 5 with a 69% accuracy using 3 input parameters and 22 output parameters in the first layer, 22 and 38 in the second, and 38 and 75 in the third respectively. The number of parameters in this set up was 292,129. 2 fully connected linear layers were utilized, including 200 learned features in the first and 10 output classes in the second. This was a 2% increase from the previous accuracy.

Illustration

